07. 성능 모니터링 및 최적화

목차

1. 성능 프로파일러
2. 메트릭 수집 및 분석
3. 자동 최적화 시스템
4. 메모리 최적화
5. CPU 최적화
6. 네트워크 최적화

성능 프로파일러

PerformanceProfiler 구조

```
pub struct PerformanceProfiler {
enabled: bool,
samples: Arc<RwLock<VecDeque<PerformanceSample>>>,
max_samples: usize,
}
```

```
pub struct PerformanceSample {
pub timestamp: Instant,
```

```
pub operation: String,
pub duration: Duration,
pub success: bool,
pub metadata: HashMap<String, String>,
}
```

성능 샘플 기록

```
impl PerformanceProfiler {
pub async fn record_operation<F, R>(&self, operation: &str, f: F) -> R
where
F: std::future::Future<Output = Result<R, Box<dyn std::error::Error + Send + Sy
{
let start = Instant::now();
let result = f.await;
let duration = start.elapsed();
let sample = PerformanceSample {
timestamp: start,
operation: operation.to_string(),
duration,
success: result.is_ok(),
metadata: HashMap::new(),
};
```

```
self.record_sample(sample).await;
match result {
Ok(value) => value,
Err(e) => panic!("Operation failed: {}", e),
}
}
}
```

메트릭 수집 및 분석

성능 통계 구조

```
pub struct PerformanceStats {
pub operation: String,
pub sample_count: usize,
pub success_rate: f64,
pub avg_duration: Duration,
pub min_duration: Duration,
pub max_duration: Duration,
pub p50_duration: Duration,
pub p95_duration: Duration,
pub p99_duration: Duration,
pub throughput_per_second: f64,
```

```
}
```

통계 계산

```
impl PerformanceProfiler {
pub  async  fn  get_stats(&self, operation: &str) -> Option<PerformanceStats> {
let  samples = self.samples.read().await;
let  operation_samples: Vec<_> = samples
.iter()
.filter(|s| s.operation == operation)
.collect();

if  operation_samples.is_empty() {
return None;
}

let  sample_count = operation_samples.len();
let  success_count = operation_samples.iter().filter(|s| s.success).count();
let  success_rate = success_count  as f64 / sample_count  as f64;

let  mut  durations: Vec<Duration> = operation_samples
.iter()
.map(|s| s.duration)
```

```
.collect();

durations.sort();


let  total_duration: Duration = durations.iter().sum();

let  avg_duration = total_duration / sample_count  as u32;

Some(PerformanceStats {

operation: operation.to_string(),

sample_count,

success_rate,

avg_duration,

min_duration: durations[0],

max_duration: durations[sample_count - 1],

p50_duration: durations[sample_count * 50 / 100],

p95_duration: durations[sample_count * 95 / 100],

p99_duration: durations[sample_count * 99 / 100],

throughput_per_second: self.calculate_throughput(&operation_samples),

})

}

}
```

자동 최적화 시스템

최적화 권장사항

```
pub struct OptimizationRecommendation {
pub category: String,
pub priority: Priority,
pub description: String,
pub suggested_config: HashMap<String, serde_json::Value>,
pub expected_improvement: f64,
}


pub enum Priority {
Low,
Medium,
High,
Critical,
}
```###

성능 최적화 분석

```rust
impl PerformanceOptimizer {
pub async fn analyze(&self) -> Vec<OptimizationRecommendation> {
let stats = self.profiler.get_all_stats().await;
let mut recommendations = Vec::new();
```

```
for stat in &stats {
recommendations.extend(self.analyze_operation_stats(stat));
}

recommendations.sort_by(|a, b| b.priority.cmp(&a.priority));
recommendations
}

fn analyze_operation_stats(&self, stats: &PerformanceStats) -> Vec<Optimization
let mut recommendations = Vec::new();

// 높은 지연시간 확인
if stats.avg_duration > Duration::from_millis(500) {
recommendations.push(OptimizationRecommendation {
category: "Latency".to_string(),
priority: Priority::High,
description: format!(
"High average latency detected for {}: {:?}",
stats.operation, stats.avg_duration
),
suggested_config: {
let mut config = HashMap::new();
config.insert("fast_path_timeout".to_string(),
```

```
serde_json::Value::String("100ms".to_string()));
config.insert("timeout_adaptation_factor".to_string(),
serde_json::Value::Number(serde_json::Number::from_f64(0.15).unwrap()));
config
},
expected_improvement: 0.3,
});
}

// 낮은 성공률 확인
if stats.success_rate < 0.95 {
recommendations.push(OptimizationRecommendation {
category: "Reliability".to_string(),
priority: Priority::Critical,
description: format!(
"Low success rate for {}: {:.2}%",
stats.operation, stats.success_rate * 100.0
),
suggested_config: {
let mut config = HashMap::new();
config.insert("byzantine_threshold".to_string(),
serde_json::Value::Number(serde_json::Number::from(2)));
config.insert("max_fast_path_timeout".to_string(),
```

```
serde_json::Value::String("10s".to_string()));

config

},

expected_improvement: 0.2,

});

}



recommendations

}

}
```

메모리 최적화

메모리 사용량 추적

```
pub struct MemoryOptimizer {

allocation_tracker: Arc<RwLock<HashMap<String, usize>>>,

}



impl MemoryOptimizer {

pub async fn track_allocation(&self, component: &str, size: usize) {

let mut tracker = self.allocation_tracker.write().await;

*tracker.entry(component.to_string()).or_insert(0) += size;
```

```
}

pub  async  fn  get_memory_recommendations(&self) -> Vec<OptimizationRecommendat
let  usage = self.get_memory_usage().await;
let  mut  recommendations = Vec::new();

let  total_usage: usize = usage.values().sum();
if  total_usage > 1024 * 1024 * 1024 { // 1GB
recommendations.push(OptimizationRecommendation {
category: "Memory".to_string(),
priority: Priority::High,
description: format!("High memory usage detected: {} MB", total_usage / (1024 *
suggested_config: {
let  mut  config = HashMap::new();
config.insert("timeout_history_size".to_string(),
serde_json::Value::Number(serde_json::Number::from(50)));
config.insert("gc_depth".to_string(),
serde_json::Value::Number(serde_json::Number::from(20)));
config
},
expected_improvement: 0.3,
});
}
```

```
recommendations
}
}
```

CPU 최적화

CPU 사용량 모니터링

```
pub struct CpuOptimizer {
cpu_samples: Arc<RwLock<VecDeque<CpuSample>>>,
max_samples: usize,
}

pub struct CpuSample {
pub timestamp: Instant,
pub component: String,
pub cpu_percent: f64,
pub thread_count: usize,
}

impl CpuOptimizer {
pub async fn get_cpu_recommendations(&self) -> Vec<OptimizationRecommendation
```

```
let samples = self.cpu_samples.read().await;
let mut recommendations = Vec::new();

if samples.is_empty() {
return recommendations;
}

let avg_cpu: f64 = samples.iter().map(|s| s.cpu_percent).sum::<f64>() / samples
if avg_cpu > 80.0 {
recommendations.push(OptimizationRecommendation {
category: "CPU".to_string(),
priority: Priority::High,
description: format!("High CPU usage detected: {:.1}%", avg_cpu),
suggested_config: {
let mut config = HashMap::new();
config.insert("max_concurrent_requests".to_string(),
serde_json::Value::Number(serde_json::Number::from(100000)));
config.insert("message_batch_size".to_string(),
serde_json::Value::Number(serde_json::Number::from(50)));
config
},
expected_improvement: 0.25,
});
```

```
}

recommendations
}
}
```

네트워크 최적화

네트워크 성능 분석

```
impl AutobahnController {
pub  async  fn  analyze_network_performance(&self) -> Result<Vec<OptimizationRec
let  mut  recommendations = Vec::new();
let  network_conditions = self.monitor_network_conditions().await?;
let  aggregation_stats = self.get_aggregation_stats().await;

// 네트워크 지연시간 분석
if  network_conditions.latency == NetworkCondition::Poor {
recommendations.push(OptimizationRecommendation {
category: "Network".to_string(),
priority: Priority::High,
description: "High network latency detected".to_string(),
suggested_config: {
```

```
let mut config = HashMap::new();
config.insert("fast_path_timeout".to_string(),
serde_json::Value::String("300ms".to_string()));
config.insert("message_batch_timeout".to_string(),
serde_json::Value::String("100ms".to_string()));
config
},
expected_improvement: 0.2,
});
}


// 메시지 집계 효율성 분석
if aggregation_stats.average_batch_size < 10.0 {
recommendations.push(OptimizationRecommendation {
category: "Aggregation".to_string(),
priority: Priority::Medium,
description: "Low message aggregation efficiency".to_string(),
suggested_config: {
let mut config = HashMap::new();
config.insert("batch_size_threshold".to_string(),
serde_json::Value::Number(serde_json::Number::from(50)));
config.insert("batch_timeout".to_string(),
serde_json::Value::String("75ms".to_string()));
```

```
config

},

expected_improvement: 0.15,

});

}


Ok(recommendations)

}

}
```

동적 설정 조정

```
impl ConfigurationTuner {

pub  fn  auto_tune(

&mut  self,

stats: &[PerformanceStats],

network_condition: NetworkCondition,

) -> Vec<TuningEvent> {

let  mut  events = Vec::new();


// 네트워크 조건에 따른 타임아웃 조정

events.extend(self.tune_timeouts(network_condition));
```

```
// 처리량에 따른 배치 매개변수 조정
events.extend(self.tune_batching(stats));

// 신뢰성에 따른 비잔틴 감지 조정
events.extend(self.tune_byzantine_detection(stats));

self.tuning_history.extend(events.clone());
events
}

fn tune_timeouts(&mut self, network_condition: NetworkCondition) -> Vec<Tuning
let mut events = Vec::new();

let (target_timeout, adaptation_factor) = match network_condition {
NetworkCondition::Excellent => (Duration::from_millis(50), 0.2),
NetworkCondition::Good => (Duration::from_millis(100), 0.15),
NetworkCondition::Fair => (Duration::from_millis(200), 0.1),
NetworkCondition::Poor => (Duration::from_millis(500), 0.05),
};

if self.current_config.timeout.base_timeout != target_timeout {
let old_value = serde_json::to_value(&self.current_config.timeout.base_timeout)
self.current_config.timeout.base_timeout = target_timeout;
```

```
let new_value = serde_json::to_value(&target_timeout).unwrap();


events.push(TuningEvent {
timestamp: std::time::SystemTime::now(),
parameter: "fast_path_timeout".to_string(),
old_value,
new_value,
reason: format!("Network condition: {:?}", network_condition),
performance_impact: None,
});
}


events
}
}
```

성능 보고서 생성

```
impl PerformanceOptimizer {
pub async fn generate_report(&self) -> PerformanceReport {
let stats = self.profiler.get_all_stats().await;
let recommendations = self.analyze().await;
```

```
PerformanceReport {
timestamp: Instant::now(),
stats,
recommendations,
summary: self.generate_summary(&stats).await,
}
}


async fn generate_summary(&self, stats: &[PerformanceStats]) -> PerformanceSum
let total_operations: usize = stats.iter().map(|s| s.sample_count).sum();
let avg_success_rate = if !stats.is_empty() {
stats.iter().map(|s| s.success_rate).sum::<f64>() / stats.len() as f64
} else {
0.0
};
let avg_throughput = if !stats.is_empty() {
stats.iter().map(|s| s.throughput_per_second).sum::<f64>() / stats.len() as f64
} else {
0.0
};


let overall_health = if avg_success_rate > 0.99 && avg_throughput > 50.0 {
HealthStatus::Excellent
```

```
} else if avg_success_rate > 0.95 && avg_throughput > 20.0 {
HealthStatus::Good
} else if avg_success_rate > 0.90 && avg_throughput > 10.0 {
HealthStatus::Fair
} else {
HealthStatus::Poor
};


PerformanceSummary {
total_operations,
avg_success_rate,
avg_throughput,
overall_health,
}
}
}
```

성능 모니터링 및 최적화 시스템은 Autobahn BFT의 지속적인 성능 향상을 위한 핵심 도구로, 실시간 분석과 자동 조정을 통해 최적의 성능을 유지합니다.